

- **Sparsity of words**
- **Scale**
- **Unmodeled representation:** context, shared understanding

What makes good NLP?

- Sensitivity to a wide range of phenomena and constraints in language
- Generality across languages, modalities, genres, styles
- Strong formal guarantees (e.g., convergence, statistical efficiency, consistency)
- High accuracy when judged against expert annotations or test data
- Computational efficiency, interpretable, ethical, ...

2 - Strings & Words

RegEx: See printout

Corpus preprocessing

Word-based tokenization

- Match all words, numbers, punctuation marks OR match `\b`
- Other approaches:



- Sequential labeling and extract based on same label.
- Language issues: e.g. Chinese is not space-separated, but longest prefix matching works well.

Sub-word based tokenization

- Why: OOV and rare words are problematic. We want to split them into known tokens.
- **BPE token learner:**
 1. Marks end of word with " _".
 2. Find the 2 tokens most frequently adjacent to each other.
 3. Merge them.
 4. Repeat until k merges have been done.
- **WordPiece:**
 - Marks continuation of a word with " _". (e.g. n _e _w _e _s _t)
 - Fundamental difference: find the 2 tokens **most likely** adjacent to each other.
 - Likelihood: $\frac{P(t_1, t_2)}{P(t_1)P(t_2)} = \frac{P(t_2|t_1)}{P(t_2)} \propto \frac{\text{count}(t_1, t_2)}{\text{count}(t_1) \cdot \text{count}(t_2)}$

-

Normalization

- Goal: convert to a canonical form. e.g. U.S.A $\rightarrow$ USA.
- Opposite: asymmetric expansion
- **Case folding:** When not to fold? NER, machine translation
- **Stemming:** Reduce words to their stem by crudely chopping affixes.
- **Lemmatization:** Reduce inflections to base form, differentiating the word forms (requires POS tags).

Sentence segmentation

- Binary classification of each period (e.g. with a decision tree)
- Other possible features: length of words before/after the period, distance to the next punctuation mark, probabilities derived from a dataset, ...

3 - n-Gram

Language models

1. Application of Chain Rule:

$$P(w_1, \dots, w_i) = P(w_1) \cdot P(w_2|w_1) \cdot \dots \cdot P(w_i|w_{1:i-1})$$

2. MLE: $P(w_i|w_{1:i-1}) = \frac{\text{count}(w_{1:i-1}w_i)}{\text{count}(w_{1:i-1})}$

- Problems: very long sequences $\rightarrow \text{count}(\cdot) = 0$

Markov assumption: n-Gram models

- Use log for small products.
- Handle OOV: special token, subword tokenization or **smoothing**.

Smoothing

Laplace smoothing

- Add 1
- $P_{\text{Laplace}}(w_n|w_{1:n-1}) = \frac{\text{count}(w_{1:n-1}w_n) + 1}{\text{count}(w_{1:n-1}) + |V|}$
- Problem: some non-zero probabilities changes a lot.
- $\text{count}^*(w_{n-1}w_n) = (\text{count}(w_{n-1}w_n) + 1) \cdot \frac{\text{count}(w_{n-1})}{\text{count}(w_{n-1}) + |V|}$
- $d_c = \frac{\text{count}^*}{\text{count}}$

Backoff & interpolation

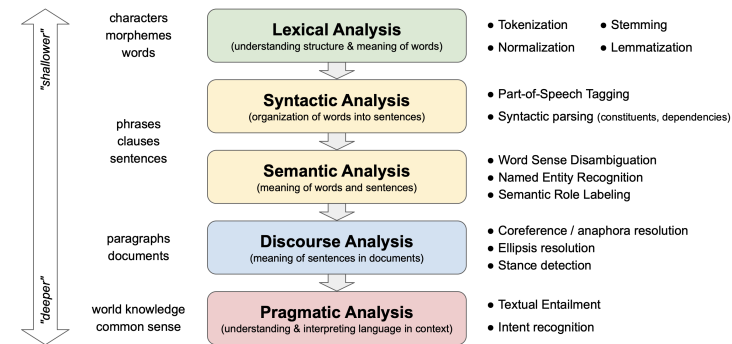
- Backoff: If trigram P not available, use bigram P.
- Interpolation: Estimate trigram P as a weighted sum of trigram P, bigram P and unigram P, with learnable weights conditioned on the context.
- Learning the weights: train-test split the corpus and find the weights that maximizes over the test corpus.

Kneser-Ney smoothing (bigram only)

$$P_{KN}(w_n|w_{n-1}) = \frac{\max[\text{count}(w_{n-1}w_n) - d, 0]}{\text{count}(w_{n-1})} + \lambda(w_{n-1})P_{KN}(w_n)$$

- Absolute discounting: removes fixed d from bigram counts (intuition: big counts are hardly affected and small counts are not useful to begin with). d set to about 0.75.
- Interpolation with a twist:
 - $P_{KN}(w)$: How likely is w to appear as a novel continuation?

1 - Introduction



- Building blocks of written language: character, morpheme (smallest meaning-bearing unit in a language, e.g. re-act-ion), word, phrase, clause (phrase with a subject and a verb), sentence, paragraph, document, corpus (collection of documents).
- Morphology: study of the forms & formation of words in a language.
- Free morphemes: (stems) can stand alone as its own word.
- Bound morphemes: derivational (change the semantic meaning or POS of the word, e.g. un-, -er as in noun, -less), inflectional (assign a grammatical property, e.g. -ed, -s, 's, -er as in comparative form)
- Fundamental tasks: lexical analysis—tokenization, syntactic analysis—POS tagging, syntactic parsing, semantic analysis—word sense disambiguation, NER, semantic role labeling (5 Whs), discourse analysis—coreference resolution (which word does "it" refer to), ellipsis resolution (e.g. She's very funny. Her sister is not (very funny).), pragmatic analysis—textual entailment (determine the inference relation between 2 ordered texts), intent recognition (e.g. "I'm hungry!": the writer is looking for a place to eat)...

Challenges

- **Ambiguity:**
 - Anaphoric ambiguity: ambiguous resolution of coreferences. (e.g. Winograd Schema: "I poured water from the bottle into the cup until it was full." vs "I poured water from the bottle into the cup until it was empty.")
 - Pragmatic ambiguity: e.g. "Do you know the time?" "Yes!" vs "It's 4.30pm."
 - Expressivity: idioms, neologisms ("Gen-Z" words), literary devices (humor, sarcasm, ...)

- $P_{KN}(w) = \frac{\# \text{ words } w' \text{ that form an existing bigram } w'w}{\text{total number of existing bigrams}}$
- Normalizing factor
 $\lambda(w_{n-1}) = \frac{d}{\text{count}(w_{n-1})} \cdot |\{w' : \text{count}(w_{n-1}w') > 0\}|$ (# words that can follow)

Evaluating language models

- Extrinsic: Use downstream task.
- Intrinsic: Evaluate each LM on a test corpus.
- 1. Train LM on a training corpus (80%)
- 2. Tune parameters with a development corpus (10%)
- 3. Evaluate on test corpus (10%)

Perplexity

- $PP(W) = \sqrt[N]{\frac{1}{P(w_1, w_2, \dots, w_N)}}$
- $\ln PP(W) = -\frac{1}{N} P(w_1, w_2, \dots, w_N)$

4 - Text Classification

Naive Bayes

- **BoW representation:** affected by tokenization and normalization.
- Independent assumption: $P(y|w_1, w_2, \dots, w_n) \propto P(w_1|y)P(w_2|y) \dots P(w_n|y)P(y) = P(y) \prod_{i=1}^n P(w_i|y)$
- $P(y) \approx \frac{N_y}{N}$ (no. of documents of class y / total no. of documents)
- $P(w_i|y) \approx \frac{\text{count}(w_i, y)}{\sum_{w \in V} \text{count}(w, y)}$ (total occurrences of w in all documents of class y / total words in documents of class y)
- Arithmetic underflow: use log
- OOV: Add-k smoothing: $\hat{P}(w_i|y) = \frac{\text{count}(w_i, y) + k}{\sum_{w \in V} \text{count}(w, y) + k|V|}$,
 $\hat{P}(y) = \frac{N_y + k}{N + k|Y|}$
- NB vs LM:
- • NB is unigram model.
- • NB treats each class like a separate language model.
- Limitation: BoW cannot handle negation (with only addition of word "not"). Mitigation: Add "not_" to all the words between negation word and next punctuation mark.

Evaluation of classifiers

- Accuracy = (TP + TN) / (all 4)
- Precision = TP / (TP + FP)
- Recall = TP / (TP + FN)
- F1 = $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$
- Why harmonic mean? Imbalanced dataset.
- Micro averaging: average the raw values across confusion matrices. (Favors bigger classes)
- Macro averaging: average the F1s across each confusion matrix.
- A 2-class classifier and a 10-class classifier both have same F1. The 10-class classifier probably does better. Reason: Consider

random guessing, 2-class $\rightarrow$ 50% correctness, 10-class $\rightarrow$ 10% correctness.

Vector space model

- DTM with binary weights: $w_{t,d} = 1$: document d contains term t .
- DTM with term frequencies: $tf_{t,d} = \#$ occurrences of term t in document d .
- • Relative importance: $w_{t,d} = 1 + \log_{10} tf_{t,d}$
- Cross-document importance: $idf_t = \log_{10} \frac{|D|}{df_t}$, df_t : # documents containing t .
- **Together:** $w_{t,d} = (1 + \log_{10} tf_{t,d}) \cdot \log_{10} \frac{|D|}{df_t}$

Document similarity

- Approach 1: dot product
- • Overly favors frequent words.
- Approach 2: cosine similarity
- • $\frac{v \cdot w}{|v| \cdot |w|}$
- Value is always +ve for document vectors since vector entries are all +ve.

5 - Connectionist ML

Generative vs discriminative classifiers

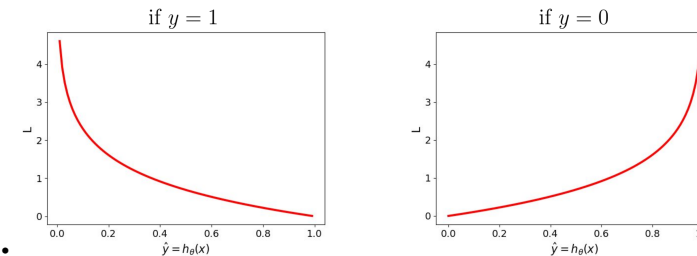
- Generative: $\hat{y} = \arg \max_{y \in Y} P(x|y)P(y)$
- Discriminative: $\hat{y} = \arg \max_{y \in Y} P(y|x)$

Logistic regression

- $\hat{y} = \frac{1}{1 + e^{-\theta^\top x}}$

Loss Function

- $L_{CE}(\hat{y}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$
- Given m samples, $L_{CE} = \frac{1}{m} \sum_j L_{CE}(\hat{y}_j, y_j)$.



- Generalized for MLR: $L_{CE}(\hat{y}, y) = -\sum_{i=1}^C y_i \log(\hat{y}_i)$, $y_i = 1$ for the correct class, 0 otherwise.

Gradient descent

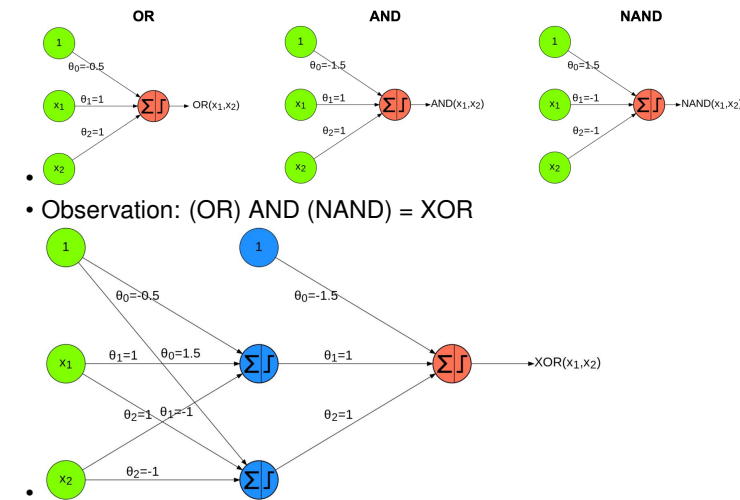
- $\frac{\partial L_{CE}}{\partial \theta_i} = \frac{1}{m} \sum_{j=1}^m [\sigma(\theta^\top x^{(j)}) - y^{(j)}] x_i^{(j)}$,
 $\frac{\partial L_{CE}}{\partial \theta} = \frac{1}{m} X^\top [\sigma(X\theta) - y]$

Regularization

- L2 ("Ridge"): $L_{CE} = \dots + \lambda \sum_{i=1}^n \theta_i^2$
- L1: $L_{CE} = \dots + \lambda \sum_{i=1}^n |\theta_i|$
- Regularization definitely increases the training loss, in the hope that testing loss will be lower.

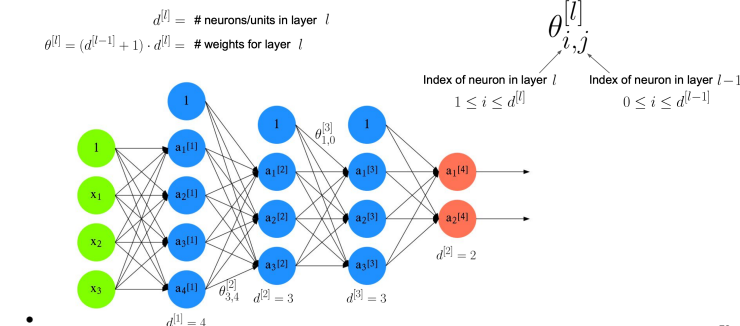
Neural networks

The XOR problem



NN

Neural Network — Indices



- **Activations:**
- • Activations for hidden layers: non-linear
- • Activations for output layers: linear for regression, sigmoid for classification.

6 - Word Embeddings and NLP Ethics

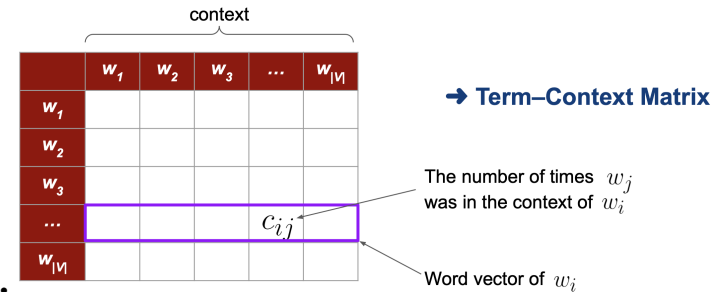
Word Embeddings

Symbolic Representations

- **One-hot encoding:** Words as discrete symbols
- Limitation: No notion of similarity between words

Sparse Word Representations

- **Distributional Hypothesis:** Words that appear in similar contexts have similar meanings.
- Revisiting Document-Term Matrix (DTM)
 - Assumption: context of w = set of documents containing w
 - Problem: does not capture distributional hypothesis
- **Term-Context Matrix (TCM)**



- Problem: skewed towards high-frequency words
- Alternative: Pointwise Mutual Information (PMI):
 $PMI(w_i, w_j) = \log \frac{P(w_i, w_j)}{P(w_i)P(w_j)}$
- Positive PMI (PPMI): $PPMI(w_i, w_j) = \max(PMI(w_i, w_j), 0)$

Assume this is the complete term-context matrix

	rating	story	data	cast	result
movie	2	4	0	1	0
show	6	3	0	2	1
nlp	0	1	3	0	4
ai	1	0	5	0	2

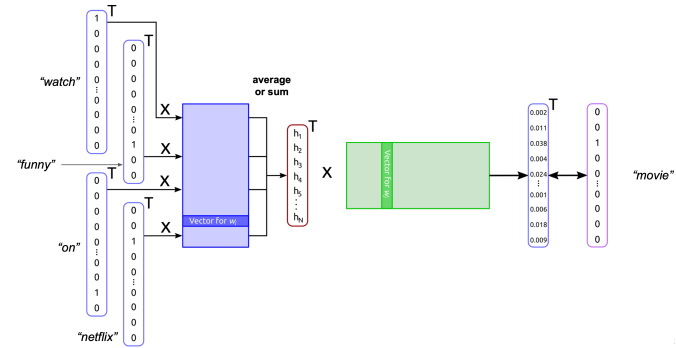
$$P(w=movie, c=cast) = 1/35 = 0.03$$
$$P(w=movie) = 7/35 = 0.2$$
$$P(c=cast) = 3/35 = 0.09$$

- Rare words: Raise context probabilities/Laplace Smoothing
- Sparsity: Very sparse

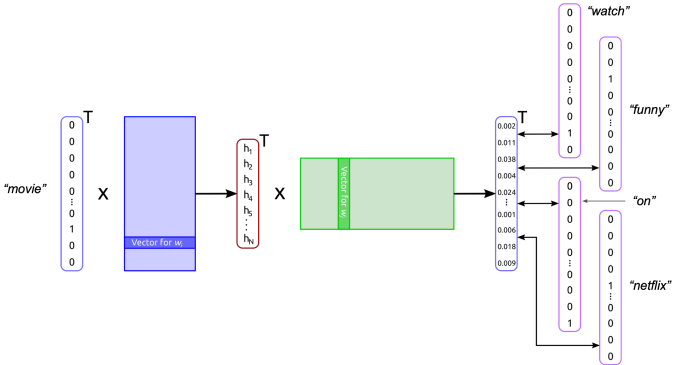
Dense Word Representations

- Basic idea: Learn a dense vector for each word, whose dimensions are "shared properties" of all the words.
- **Word2Vec**

Word2Vec — CBOW (window size m=2)



Word2Vec — Skip-Gram (window size m=2)



- Goal of training (CBOW):
 $L = -\log P(w_c | w_{c-m}, \dots, w_{c-1}, w_{c+1}, \dots, w_{c+m}) = -\log P(u_c | \tilde{v}), \tilde{v} = \sum_{-m \leq j \leq m, j \neq 0} v_{c+j}$
 $P(u_c | \tilde{v}) = \frac{\exp(u_c^T \cdot \tilde{v})}{\sum_{j=1}^{|V|} \exp(u_j^T \cdot \tilde{v})}$
- Goal of training (Skip-Gram): $\Pr(u_{c+j} | v_c) = \frac{\exp(u_{c+j}^T \cdot v_c)}{\sum_{j=1}^{|V|} \exp(u_j^T \cdot v_c)}$, "next-word prediction" task.
- By-product: Vectors of words with similar contexts are close.
- Word vectors: u or v or an average of both.

x

y

watch funny on netflix	movie
------------------------	-------

→ 1 CBOW sample

x

y

movie	watch
movie	funny
movie	on
movie	netflix

- → 2m Skip-gram samples
- **Skip-Gram with Negative Sampling (SGNS)**
 1. Produce $2m$ positive samples from the input string.
 2. Produce $2mk$ negative samples from V .
 - Penalize common words:

- $$P_{\alpha}(w_i) = \frac{Count(w_i)^{\alpha}}{\sum_{j=1}^{|V|} Count(w_j)^{\alpha}}, 0 < \alpha \leq 1$$
 3. $P(+|c, m) = \frac{1}{1 + \exp(-u_m^T v_c)} = \sigma(u_m^T v_c), P(-|c, m) = 1 - P(+|c, m)$
 4. $L = - \left[\sum_{(c,m) \in B_+} \log \sigma(u_m^T v_c) + \sum_{(c,m) \in B_-} \log \sigma(-u_m^T v_c) \right]$
 - Hyperparameters:
 - Sampling method to generate negative samples.
 - Number of negative samples k (5-20 for small text, 2-5 for large text).
 - Limitations:
 - Unable to represent phrases.
 - Unable to handle polysemy (multiple meanings) and part of speech.
 - Practical considerations:
 - Skip-gram slower, better for infrequent words. CBOW faster.
 - Hierarchical softmax better for infrequent words, negative sampling better for frequent words, better with low dimensional vectors.

NLP Ethics

Biases

- Bias in word embeddings: e.g.
 $v(programmer) - v(man) + v(woman) \approx v(homemaker)$
- Bias in sentiment analysis: e.g. different sentiment scores for different cuisine.

Towards debiasing

- **Measuring bias:** e.g. find pairs of (x, y) that minimize $\cosine(v(she) - v(he), v(x) - v(y))$.
- **Identify gender subspace:** Pick top pairs of gender words, compute difference vectors, use top PCs from PCA as gender subspace.
- Train binary classifier to identify gender-neutral/specific words.
- Embedding transformation:
 - Approach: Find T by minimizing

$$\min_T \| (TW)^T (TW) - (W^T W) \|_F^2 + \lambda \| (TN)^T (TB) \|_F^2$$

inner products of embeddings after transformation

inner products of embeddings before transformation

transformed gender-neutral word vectors

transformed gender subspace

Keep difference (here: Frobenius norm) small — preserve the original embeddings as much as possible!

Minimal if gender subspace removed from vectors of gender-neutral words

- $\|A\|_F^2 = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}$
- Generate morphological variations (grammatically incorrect) data to desensitize the model.

7 - Sequences

POS Tagging

- Two categories:
 - Closed class: small, fixed set of words (e.g. prepositions, conjunctions)
 - Open class: large, open set of words (e.g. nouns, verbs)

Baseline: Label with most frequent tag

- Limitations:
 - Imbalanced: High accuracy for common unambiguous words, which are not interesting for downstream tasks.
 - Downstream error propagation: Errors in POS tagging quickly propagate up.

Unsupervised Algorithms

- Anchor words: Words with unambiguous POS tags.

Supervised Algorithms: Hidden Markov Models

- Notations:
 - The HMM $\theta = (A, B, \pi)$
 - Finite set of states (**POS tags**) $S = \{s_1, s_2, \dots, s_N\}$
 - Sequences of states $Q = q_1, q_2, \dots, q_T, q_t \in S$
 - Finite set of observations (**words**) $V = \{v_1, v_2, \dots, v_M\}$
 - Sequences of observations $O = o_1, o_2, \dots, o_T, o_t \in V$
 - Transition probabilities $a_{ij} = P(q_t = s_j | q_{t-1} = s_i)$
 - Emission probabilities $b_i(o_k) = P(o_t = v_k | q_t = s_i)$
 - Initial state probabilities $\pi_i = P(q_1 = s_i)$
- Core tasks:
 - Model learning: Given Q and O , learn A, B, π .
 - Likelihood: Given θ, Q and O , compare $P(O|Q, \theta)$.
 - Decoding: Given θ and O , find Q .

Model Learning

- $\pi_i = P(q_1 = s_i) = \frac{\text{Count}(<S>s_i)}{\text{Count}(<S>)}$
- $a_{ij} = P(q_t = s_j | q_{t-1} = s_i) = \frac{\text{Count}(s_i s_j)}{\text{Count}(s_i)}$
- $b_i(o_k) = P(o_t = v_k | q_t = s_i) = \frac{\text{Count}(v_k, s_i)}{\text{Count}(s_i)}$

Likelihood

- $P(O, Q|\theta) = \prod_{t=1}^T P(o_t|q_t)P(q_t|q_{t-1}), P(q_1|q_0) = P(q_1)$

Decoding

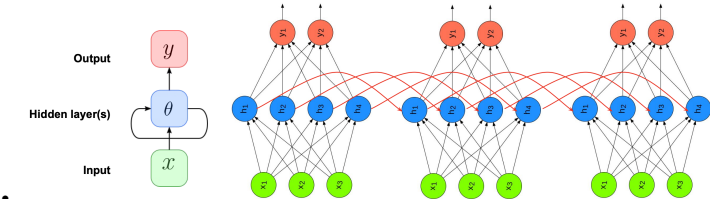
- Naive decoding: Enumerate all possible Q_i and find $\arg \max_{Q_i} P(O, Q_i|\theta)$
- Runtime complexity: $O(|Q|^T \cdot T)$ (T for computing joint probability)
- Viterbi Algorithm:
 - For every state s_j at time t , compute the probabilities of all inward paths ($O(|Q|)$) and keep the path with the highest probability.
 - Runtime complexity: $O(|Q|^2 \cdot T)$

- Recursion:
 - $v_t(j) = \max_{i=1}^{|Q|} v_{t-1}(i) a_{ij} b_j(o_t)$
 - $bt_t(j) = \arg \max_{i=1}^{|Q|} v_{t-1}(i) a_{ij} b_j(o_t)$
- Practical issue: Arithmetic underflow, use log probabilities.

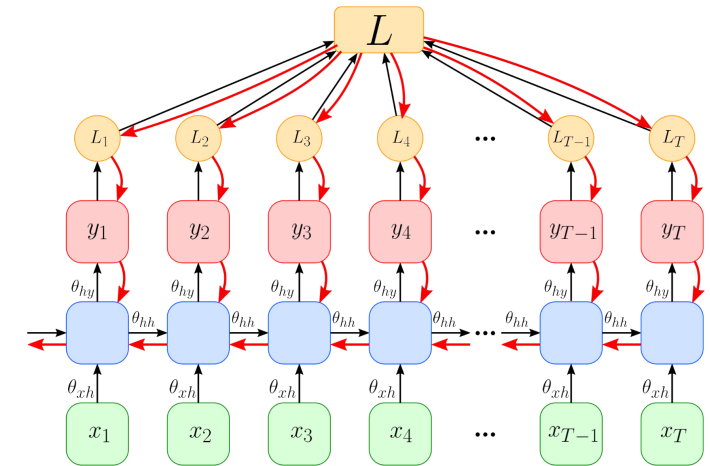
8 - Encoder-Decoder

RNNs

- Autoregressive Generation:** Sample further words conditioned on previous choices until: reaching a pre-determined length or an end-of-sequence token is generated.
- Basic recurrent formula:** $h_t = f_{\theta}(h_{t-1}, x_t)$ e.g. $h_t = \tanh(\theta_{hh} h_{t-1} + \theta_{xh} x_t)$



- Loss calculation and backpropagation (BPTT):
- Important:** parameter-tied: only one copy of θ_{hh}, θ_{xh} and θ_{hy} .



- Problems with BPTT: vanishing gradients and time complexity. NOT memory complexity.
- Mitigation: windowing: only backpropagate through a fixed number of time steps. The window size depends on VRAM limits.

How can we try to mitigate these problems with BPTT?

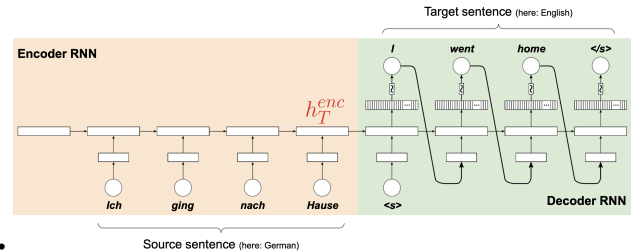
The diagram shows an encoder-decoder RNN. The encoder (left) processes input $x_1, x_2, x_3, x_4, \dots, x_{T-1}, x_T$ through hidden states $h_1, h_2, h_3, h_4, \dots, h_{T-1}, h_T$ to produce output L . The decoder (right) takes L as input and produces output $y_1, y_2, y_3, y_4, \dots, y_{T-1}, y_T$ through hidden states $h_1, h_2, h_3, h_4, \dots, h_{T-1}, h_T$.

- A Use less precision
- B Cap losses ✓
- C Bring back Markov! ✓
- D Skip Some Connections ✓

Conditional RNNs

Encoder-decoder

- Encoder: Maps the **context** into a fixed-length vector c .
- Decoder: Maps the c into a sequence of words.
- Kalchbrenner and Blunsom (2013):**
 - $h_t = \sigma(\theta_{hh} h_{t-1} + \theta_{xh} x_t + s)$, $y_t = \text{softmax}(\theta_{hy} h_t)$
- Sutskever et al. (2014):**
 - $h_t^{enc} = \sigma(\theta_{hh}^{enc} h_{t-1}^{enc} + \theta_{xh}^{enc} x_t)$, y_t^{enc} is not computed.
 - $h_t^{dec} = \sigma(\theta_{hh}^{dec} h_{t-1}^{dec} + \theta_{xh}^{dec} x_t)$, $y_t^{dec} = \text{softmax}(\theta_{hy}^{dec} h_t^{dec})$
 - IMPT:** $h_0^{dec} = h_T^{enc}$



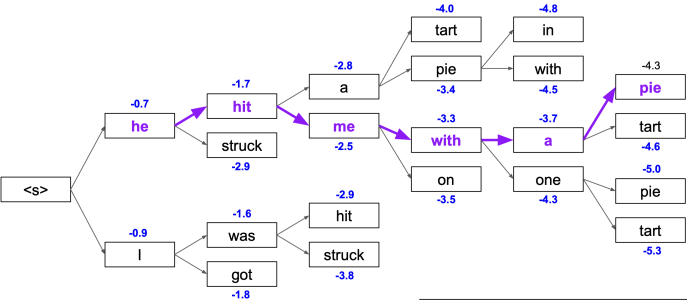
Attention

- Motivation:** c (e.g. h_T^{enc}) is used to represent the entire input sequence (**bottleneck**).
- 1. Attention score (using dot product here): $e = [h_t^T h_s^{(1)}, h_t^T h_s^{(2)}, \dots, h_t^T h_s^{(N)}] \in \mathbb{R}^N$
- 2. Attention weights: $\alpha = \text{softmax}(e) \in \mathbb{R}^N$
- 3. Context vector: $c_t = \sum_{i=1}^N \alpha_i h_s^{(i)} \in \mathbb{R}^H$
- 4. (vanilla) $y_t = \text{softmax}(\theta_{hy}[c_t, h_t])$
- "Query" h_t is the decoder hidden state. "Key"/"Value" $h_s^{(i)}$ is the encoder hidden state.

Beam search decoding

- Motivation: Local optimal $\neq$ global optimal.
- Idea: Keep track of the top k most likely sequences at each time step.
- Hypothesis: $Y_i = \{y_1, y_2, \dots, y_t\}$
- score $(Y_i) = \log P(y_1, y_2, \dots, y_t | x)$

Example



- Different hypothesis produces </s> at different steps.
- Stops when:
 - A maximum of T steps is reached,
 - At least n hypotheses have produced </s>.
- Sampling:
 - Pure sampling: Sample k words from the entire vocabulary based on the probability distribution.
 - Top- m sampling: Sample k words from the subset of the vocabulary consisting of m most probable words.

9 - Trees

Syntactic parsing

- Constituent: group of words that behaves as a single unit. Sentences can be described as a hierarchical structure of constituents.
- Constituent tests:
 - Topicalization: can be moved around
 - Proform substitution: can be substituted with it/that/them/then/there, etc.
- Fragment answers: can answer a question while retaining the meaning of the original sentence.

Context-free grammars

special start symbol $\rightarrow S \rightarrow NP VP$
 $NP \rightarrow Det Noun$
 $VP \rightarrow Verb NP$

Example

$Det \rightarrow a \mid the$
 $Noun \rightarrow man \mid meal \mid flight$
 $Verb \rightarrow saw \mid booked$
set of rules or productions

- Definition: a 4-tuple $(N, \sum, R, S)$
- N - set of non-terminal symbols
- $\sum$ - set of terminal symbols

- R - set of rules
- S - start symbol
- **Requirements:** single non-terminal on LHS, application of rule does not depend on context.

Structural ambiguity

- Attachment ambiguity, coordination ambiguity.

Chomsky Normal Form

- Change in rules: only allow 1 terminal or 2 non-terminals on RHS.
- Transform from CFG to CNF:
 - Recursive removal of unary/empty rules: e.g. Nominal $\rightarrow$ Noun, Noun $\rightarrow$ book: Nominal $\rightarrow$ book, Noun $\rightarrow$ book.
 - Dividing n-ary rules by introducing new non-terminals: e.g. $S \rightarrow Aux NP VP$: $S \rightarrow X VP$, $X \rightarrow Aux NP$.
 - Weak equivalence: generate the same set of sentences, but derivations may differ.
 - Strong equivalence: identical expressiveness + identical derivations.

CYK Parsing

I	book	the	flight	through	Singapore
[0,1] Pronoun, NP	[0,2] S	[0,3]	[0,4] S	[0,5]	[0,6] S
	[1,2] S, Nominal, Noun, Verb	[1,3]	[1,4] S, VP, X2	[1,5]	[1,6] S, VP, X2
		[2,3] Det	[2,4] NP	[2,5]	[2,6] NP
			[3,4] Nominal, Noun	[3,5]	[3,6] Nominal
				[4,5] Prep	[4,6] PP
					[5,6] PropNoun, NP

- Valid sentence: "s" in the top right cell.
- Syntactic vs semantic (e.g. a "Noun" can be replaced by any noun)

An efficient form of a recursive top-down algorithm is the Earley parsing algorithm, which is still recursive but requires the algorithm to keep state; i.e., keep track of partially completed productions. This allows it to find a valid parse in some cases asymptotically faster than the $O(n^3)$ time required for CYK.

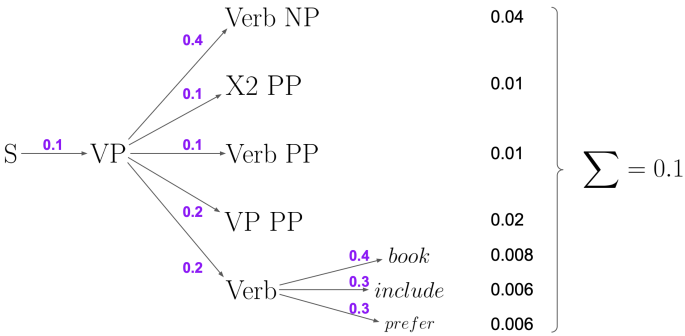
Get all parse trees Very inaccurate pseudo code:
function CKY-Parse(words, grammar) returns table, pointer

```
for j ← from 1 to LENGTH(words) do
  for all { A | A → words[j] ∈ grammar }
    table[j-1, j] ← table[j-1, j] ∪ A
    pointer[j-1, j, A] ← pointer[j-1, j, A] ∪ words[j]

for j ← from j-2 down to 0 do
  for k ← i+1 to j-1 do
    for all { A | A → BC ∈ grammar and B ∈ table[i, k] and C ∈ table[k, j] }
      table[i, j] ← table[i, j] ∪ A
      pointer[i, j, A] ← pointer[i, j, A] ∪ ((i, k, B), (k, j, C))
```

Statistical parsing

- Requirement for valid probabilities: $\sum_{\alpha} P(A \rightarrow \alpha) = 1$ (i.e. the sum of P for all rules with the same LHS is 1)
- Learning the probabilities: $P(A \rightarrow \alpha) = \frac{\text{count}(A \rightarrow \alpha)}{\text{count}(A)}$
- Converting PCFG to CNF:
 - n-ary rules: $S \rightarrow Aux NP VP$ [0.1]: $S \rightarrow X1 VP$ [0.1], $X1 \rightarrow Aux NP$ [1.0]
 - Removing unary rules:



Evaluate parse trees

Tuples only present in correct tree	Tuples resnt in both trees	Tuples only present in computed tree
$\langle NP, 3, 6 \rangle$	$\langle NP, 1, 1 \rangle$ $\langle Pronoun, 1, 1 \rangle$ $\langle VP, 2, 2 \rangle$ $\langle Verb, 2, 2 \rangle$	$\langle VP, 2, 4 \rangle$
$\langle Nominal, 4, 6 \rangle$	$\langle Det, 3, 3 \rangle$ $\langle Nominal, 4, 4 \rangle$ $\langle Noun, 4, 4 \rangle$ $\langle Prep, 5, 5 \rangle$	$\langle NP, 3, 4 \rangle$
	$\langle ProperNoun, 6, 6 \rangle$ $\langle PP, 5, 6 \rangle$ $\langle NP, 6, 6 \rangle$	

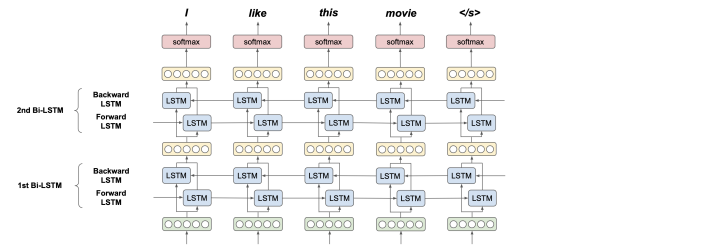
$$\text{Precision} = \frac{TP}{TP + FP} = \frac{11}{11 + 2} = 0.85$$
$$\text{Recall} = \frac{TP}{TP + FN} = \frac{11}{11 + 2} = 0.85$$
$$f1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = 0.85$$

TP = # tuples in both trees
 FP = # tuples only in the computed tree
 FN = # tuples only in the correct tree

10 - Transformers

- Motivation: Word representations should vary depending on the context. The context includes the whole sentence and the word order.

ELMo



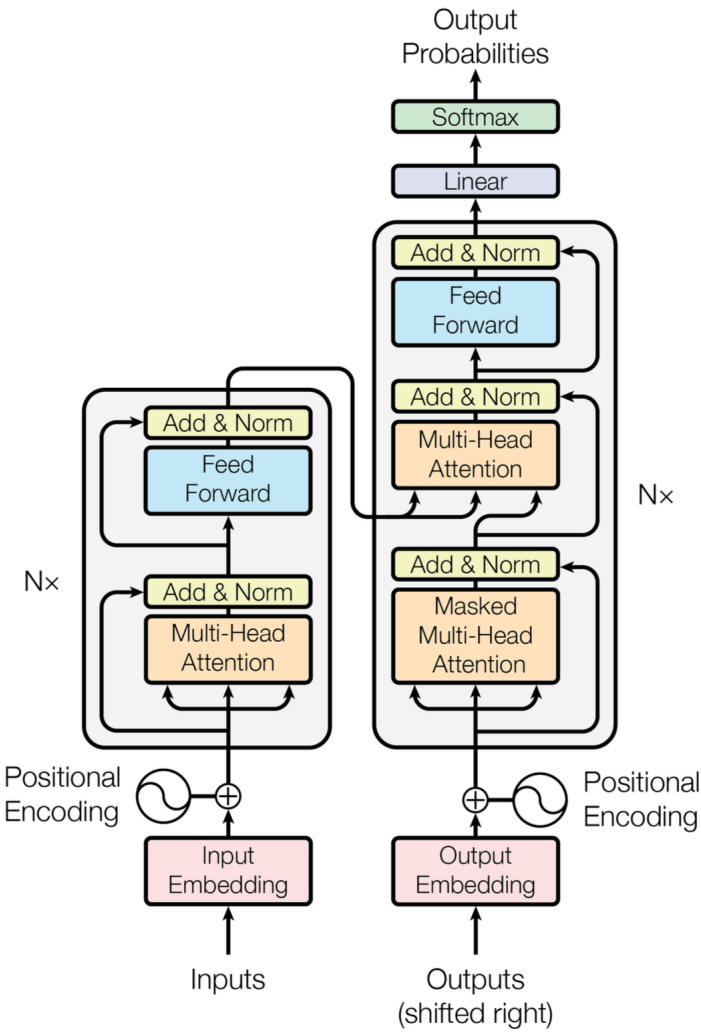
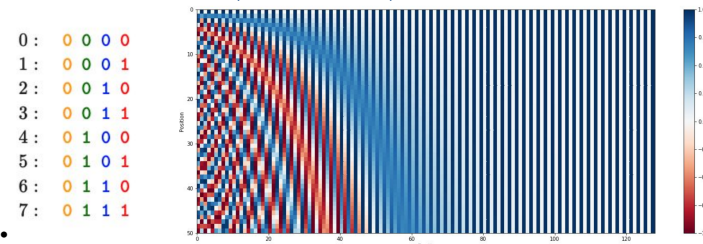
- **Two layers of bi-LSTM:** After the first layer contextualize the embeddings, the second layer **re-contextualize** the embeddings. (e.g. "traffic" in "traffic light" learns its embeddings after the first layer, and "light" learns its embeddings based on the contextualized embeddings of "traffic" in the second layer.)
- **The final embedding** is usually a weighted sum of the two layers.

Transformers

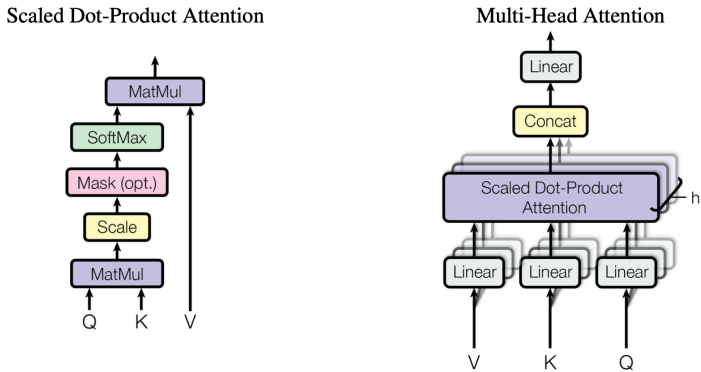
- Encoder-decoder architecture without recurrence.

Positional Encoding

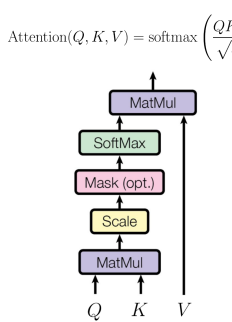
- **RNNs** process words sequentially, but **transformers** has no built-in notion of word order.
- Naive approaches include using the actual position (which quickly dominates the word embedding) or using $\frac{pos}{N-1}$ (encoding of the same position differs for different sentence lengths).
- Accepted approach: $PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$,
 $PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$



Encoder



• **Attention:** $Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$,

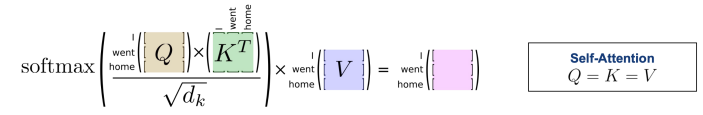


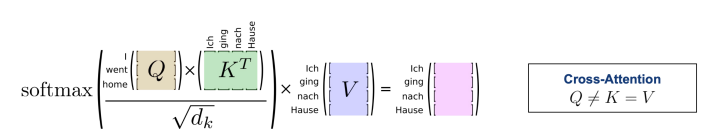
```
1 import torch
2 import torch.nn as nn
3
4
5 class Attention(nn.Module):
6     """Implements Scaled Dot Product Attention"""
7
8     def __init__(self):
9         super().__init__()
10
11     def forward(self, Q, K, V, mask=None, dropout=None):
12         # All input shapes: (batch_size B, seq_len L, model_size D)
13         # Perform QK^T (* is the dot product here)
14         # We have to use torch.matmul since we work with batches!
15         out = torch.matmul(Q, K.transpose(1, 2)) # => shape: (B, L, L)
16
17         # scale alignment scores
18         out = out / (0.5 ** 0.5)
19
20         # Push through softmax layer
21         out = f.softmax(out, dim=-1)
22
23         # Multiply scaled alignment scores with values V
24         return torch.matmul(out, V)
```

- **Attention Head:**
 - Maps model size d_{model} to d_q , d_k and d_v .
 - Proposed: $d_q = d_k = d_v = d_{model}/h$.
- **Multi-Head Attention:** The linear layer "unifies" all heads into one. $nn.Linear(h \times d_q/d_k/d_v, d_{model})$
- **Feed Forward:** $d_{model} \rightarrow d_{hidden} \rightarrow d_{model}$

Decoder

- 2 MHA blocks: one for self-attention and one for cross-attention.
 - Example: German-to-English machine translation



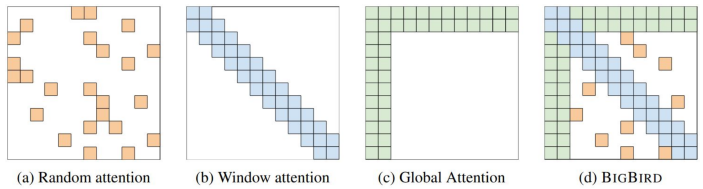


Masking

- Purpose: Ignore <PAD>, "hidden" words and future words.
- Mask "hidden" words: Train the model to predict the hidden words.
- **Mask future words:** "Causal masking" to prevent the model from seeing future words, in autoregressive training.

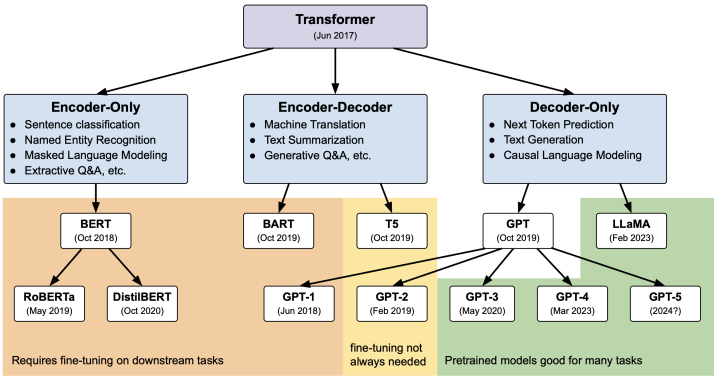
Restricted attention

- **Sparse attention:** only attend to a subset of the input sequence, to make the number of computations $O(n)$.



- Other approaches include:
 - Linear attention: Approximates attention with lower-rank representation.

- Flash attention: ...
- Selective routing: Attend to most relevant tokens.



11 - LLMs

Encoder-only

BERT

- Uses only the transformer encoder, self-supervised training.
- Trained on 2 tasks: Masked language modeling (MLM) and next sentence prediction (NSP, a *binary classification* task).

Comparison	BERT October 11, 2018	RoBERTa July 26, 2019	DistilBERT October 2, 2019	ALBERT September 26, 2019
Parameters	Base: 110M Large: 340M	Base: 125 Large: 355	Base: 66	Base: 12M Large: 18M
Layers / Hidden Dimensions / Self-Attention Heads	Base: 12 / 768 / 12 Large: 24 / 1024 / 16	Base: 12 / 768 / 12 Large: 24 / 1024 / 16	Base: 6 / 768 / 12	Base: 12 / 768 / 12 Large: 24 / 1024 / 16
Training Time	Base: 8 x V100 x 12d Large: 280 x V100 x 1d	1024 x V100 x 1 day (4-5x more than BERT)	Base: 8 x V100 x 3.5d (4 times less than BERT)	[not given] Large: 1.7x faster
Performance	Outperforming SOTA in Oct 2018	88.5 on GLUE	97% of BERT-base's performance on GLUE	89.4 on GLUE
Pre-Training Data	BooksCorpus + English Wikipedia = 16 GB	BERT + CCNews + OpenWebText + Stories = 160 GB	BooksCorpus + English Wikipedia = 16 GB	BooksCorpus + English Wikipedia = 16 GB
Method	Bidirectional Transformer, MLM & NSP	BERT without NSP, Using Dynamic Masking	BERT Distillation	BERT with reduced parameters & SOP (not NSP)

- Model distillation: Task the student model to predict the entire probability distribution of the teacher model output.

Encoder-decoder

T5

- Multi-task learning: model training on multiple tasks simultaneously (e.g., machine translation, coreference resolution, text summarization, sentence acceptability judgment, sentiment analysis)
- Each task is (re-)formulated as text-to-text task to match the encoder-decoder architecture (adding task-specific prefixes).

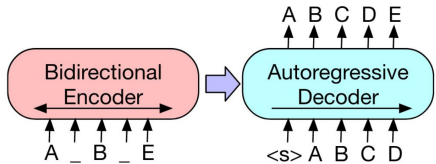
BART

- Trained by corrupting documents and then optimizing a reconstruction loss, i.e. denoising (the cross-entropy between the decoder's output and the original document)

- "Multi-task via augmentation": e.g. token masking, sentence permutation, document rotation, etc.

BART

- Arbitrary noise transformation (not just BERT-like masking)
- Bidirectional encoding + auto-regression word prediction



Decoder-only

GPT

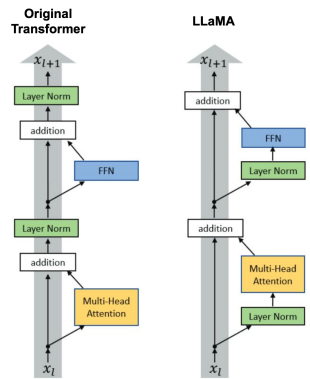
- Uses only the transformer decoder, autoregressive training.
- ChatGPT-1/2/3: text only, "just" making it larger; GPT-4: multimodal.
- ChatGPT-3+: reinforcement learning from human feedback (RLHF).
- RLHF: Use human-generated responses to prompts to fine-tune the pretrained model.

LLaMA

- Pre-normalization: layer normalization is put inside the residual blocks.

LLaMA — Pre-Normalization

- Post vs. pre-normalization
 - Post: layer normalization **between** residual blocks (original transformer)
 - Pre: layer normalization **inside** residual blocks (LLaMA, etc.)
- Observed benefit of pre-normalization:
 - Well-behaved gradients at initialization
 - Significantly faster training



- SwiGLU (Swish-Gated Linear Unit) activation: non-monotonic, parameterized activation function.
- Rotary Positional Embeddings (RoPE): encode word positions by rotating word embedding vectors.
- Open LLM: Trained exclusively on publicly available data.

Limitations and challenges

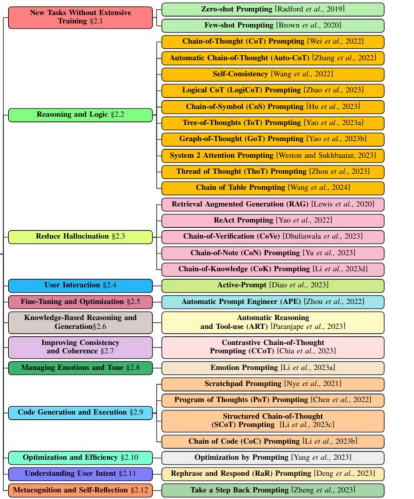
- Inferencing cost: Image generation is the highest.
- Model alignment: Aligns with users' preferences and moral compass.
- Accuracy/hallucination, Misinformation/disinformation/fake news, Prompted-based jailbreaking (e.g. pretending).

Prompt engineering

- Definition: The practice of designing and optimizing prompts to elicit desired responses from LLMs.

- Best practices (OpenAI)
 - Use the latest model
 - Put instructions at the beginning of the prompt and clearly separate instructions and prompt
 - Be specific, descriptive and as detailed as possible (about the desired context, outcome, length, format, style, etc.)
 - Start with zero-shot, then few-shot (if all fails: fine-tune)
 - Reduce "fluffy" and imprecise descriptions
 - Instead of saying what **not** to do, say what to do instead
 - Code Generation Specific – use "leading words" to nudge the model toward a particular pattern

- X-shot prompting:** zero-shot, one-shot, few-shot → In-context learning (ICL).
- In-context learning:**
 - Intuition: Demonstrations help to "locate" latent concepts acquired during pre-training.
 - Observation 1: Correctness of the demonstration label is not important.
 - Observation 2: More demonstrations help up to a certain threshold.
 - Observation 3: Relevance of demonstrations is important.
 - Observation 4: Label space matters.
 - Observation 5: Order of demonstrations and distribution of labels matter.
- RAG:**
 - Retrieve relevant snippets of knowledge (chunks) and add as context to prompt.
 - Benefits: Grounding of LLM responses on (hopefully) factual data, simple integration of (very) recent and/or very domain-specific data, improved transparency, customization/personalization.
 - Chunking:** External data needs to be splitted into "meaningful" chunks of practical sizes.
 - Fixed size chunking.
 - Recursive chunking: top-down approach to keep paragraphs/sentences intact as much as possible.
 - Document-based chunking: utilize document structure: Markdown headings, tables, source code, etc.
 - Semantic chunking: chunk = group of sentences based on their embedding similarities.
 - Agentic chunking: novel idea: let the LLM decide where to best split.
 - Storing and retrieval:**



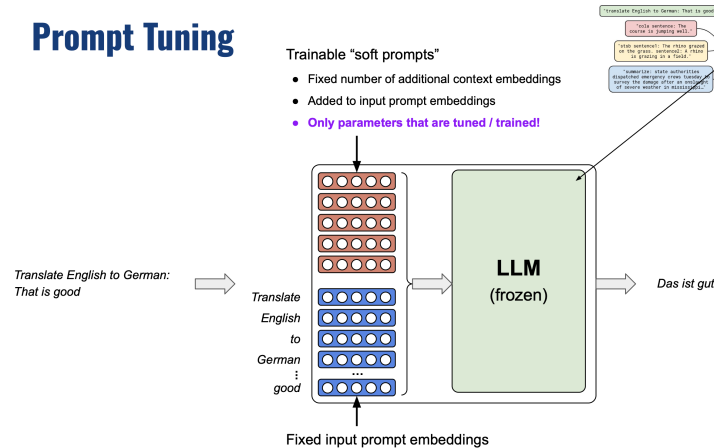
- "Classic" IR: Store chunks as text documents, index (text→document) and query with prompts.
- Vector database retrieval: Embed chunks and store embedding vectors. Index the vectors. Embed prompt and search for nearest vectors.
- Common goals: Store large volumes and support fast access.

Fine-tuning

Prompt tuning

- Automatically learn the context for a given task.
- Approach: context = trainable embedding vector

Prompt Tuning



Prefix tuning

- Add small number of trainable vectors to **each transformer block**.
- Different additional vectors for different tasks (can be swapped out during inferencing).

LoRA

- Mismatch: training objective (next-word prediction) vs. user's objective (task-specific).
- Common data setup: Instruction fine-tuning: custom instruction dataset with (instruction, output)-pairs.
- Parameter-efficient fine-tuning: only train a small number of parameters.
- **LoRA**: Low-Rank Adaptation
- Adapter: small, trainable module added to a pretrained network model. Low-rank approximation of the weight matrix.
- **Why does it work?** Weight matrices often reside within low-rank spaces, hence smaller matrices would suffice. (e.g. explained variance remains high after dimensionality reduction).
- Benefits:
 - Flexibility: LoRA can be applied to all or only some weight matrices.

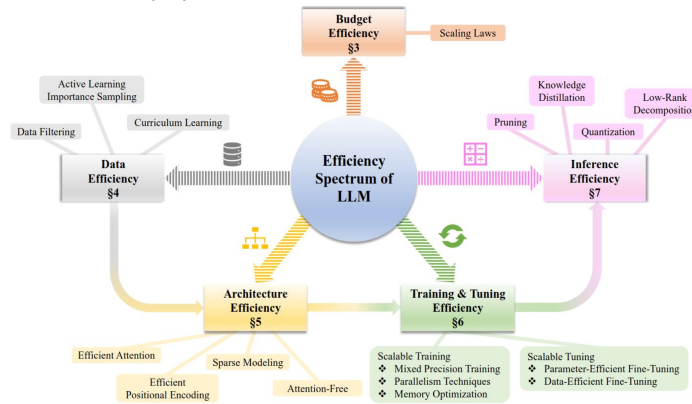
- Less trainable parameters/weights: Lower memory requirements, faster training.
- Pretrained weights remain unchanged: Preserving original model's quality.
- Drawbacks:
 - Increased complexity: Adding LoRA to existing model is not trivial.
 - Performance: LoRA might not perform as well as full fine-tuning.

Data preparation

- Irrelevant data: HTML skeletons, ...
- Low-quality data: GPT-2 approach: crowdsourcing of quality control.
- Duplicate data: e.g. repeated news articles, ...
- **Data decontamination**:
 - Often not clear which data a non-public LLM was trained on.
 - No guarantee that a test dataset was not part of the training dataset.
 - GPT-2 approach: Remove Wikipedia articles from the training dataset. (Wikipedia is commonly used for testing.)
 - Toxicity and biases: "improper" data, "Model Collapse" (when the training data has content generated by the model itself), ...
- Privacy-sensitive information

12 - Recent Developments in NLP

- The efficiency spectrum:

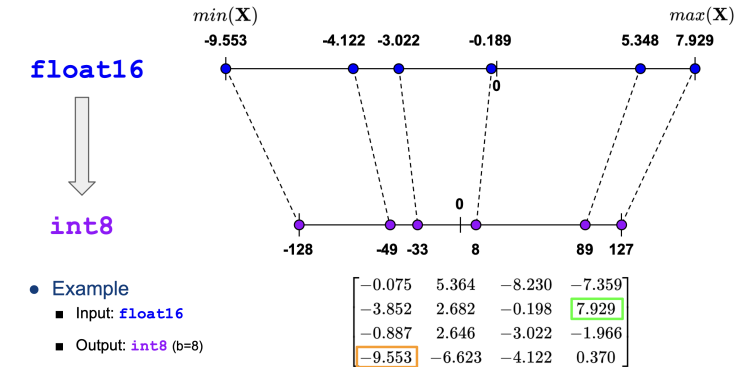


- More params → better model, but: *increased* memory requirements, training & inferencing time, energy consumption, need for more data, ...
- Scaling laws:
 - Bigger not always better.
 - Model performance follows a power law improvement with increasing #params. (insufficient data)
 - Performance scales predictably with compute until hits hardware/data limits.

Quantization

- Core idea: Reduce the numerical precision of the model weights and activations (Mostly **PTQ**: post-training quantization).
- Advantages: reduced memory requirements, faster calculations.
- Disadvantages: risk of accuracy degradation, trickier to implement (not suitable for all layers).
- **Goals**:
 - Map high-precision values to low-precision values of a smaller range.
 - Preserve overall distribution of the values.

Asymmetric quantization



$$s = \frac{2^b - 1}{\max(\mathbf{X}) - \min(\mathbf{X})} = \frac{255}{7.929 - (-9.553)} = 14.586$$

$$z = -\text{round}(\min(\mathbf{X}) \cdot s) - 2^{b-1} = -\text{round}(-9.533 \cdot 14.586) - 128 = 11$$

- Quantization: $X_{\text{quant}} = X \cdot s + z$
- Dequantization: $X_{\text{dequant}} = (X_{\text{quant}} - z)/s$
- **Handling outliers**:
 - Clipping: clip the values to a certain range.
 - Calibrate the clipping range:
 - static calibration: find best ranges before inferences using a calibration dataset
 - dynamic calibration: after each layer, collect statistics of activations to find best ranges

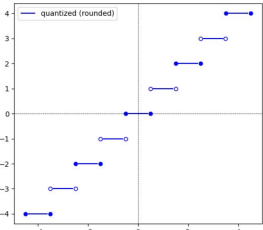
Quantization-aware training (QAT)

- Idea: Simulate quantization during training.
- **FakeQuant nodes** between the layers: quantize and immediately dequantize the values.
- Model is still trained with high-precision values under the hood.

QAT — Straight-Through Estimator (Intuition)

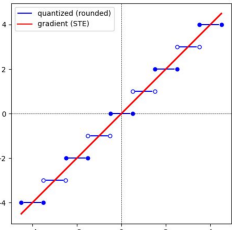
Quantization function; here only `round()`

- Fundamentally a step function
- Not (nicely) differentiable



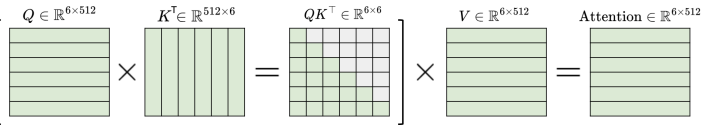
Straight-Through Estimator (STE)

- Treats (de-)quantization function as identity function (within clipping range)
- Derivative: 1 in clipping range, 0 otherwise

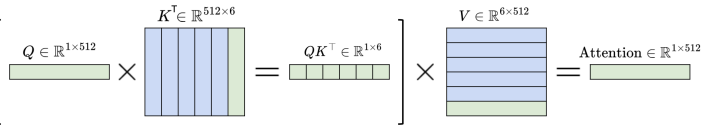


KV caching

Without caching (default)



KV caching



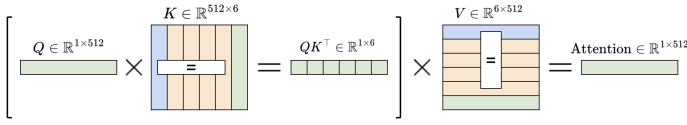
Cache size formula: $2 \cdot n \cdot b \cdot t \cdot \underbrace{(d \cdot h)}_{d_{model}} \cdot p$

Example (fp16): $2 \cdot 32 \cdot 32 \cdot 500 \cdot (128 \cdot 32) \cdot 2 \approx 7.81\text{GB}$

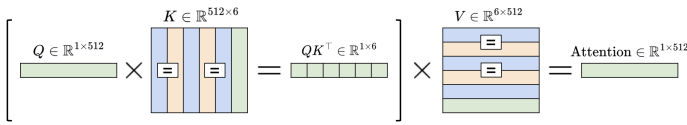
Memory optimization:

- Basic strategies: Reduce model size, reduce precision (both potentially reduce accuracy); reduce batch size (reduces efficiency).
- Advanced strategies: attention variants
- Grouped-query/multi-query attention:

Multi-Query Attention (visualized)

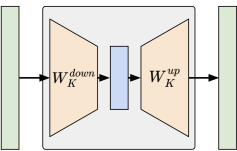


Grouped-Query Attention (visualized)



Multi-head latent attention:

- MLA – basic idea
 - Transform key and values vectors into a lower-dimensional space and back (recall: LoRA)
 - Store only lower-dimensional representation of vectors in KV cache
 - Simplified illustration (here for key vectors):



$$W_K^{down} \in \mathbb{R}^{d \times r}$$
$$W_K^{up} \in \mathbb{R}^{r \times d}$$

- only representation that is added to KV cache

Mixture of experts

Mixture of Experts

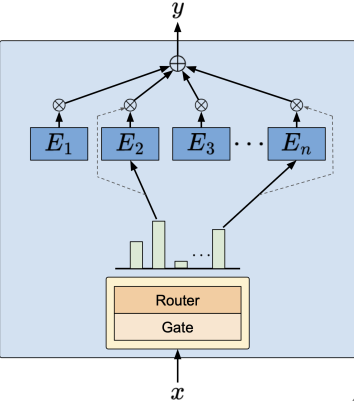
- MoE components
 - Experts $E_1, E_2, E_3, \dots, E_n$
 - Gating mechanism returning a gate G_i probability for expert E_i
 - For input x : $G(x)_i \in [0, 1]$

Can be 0!

MoE output

- Weighted sum of outputs of all involved experts

$$y = \sum_{i=1}^n G(x)_i E_i(x)$$



- Expert: Architecture can be anything (even a mixture of different architectures).
- Gate: e.g. a FFNN that returns logits/softmax probabilities.
- Router: subnetwork that decides which experts to pass the input to based on the gate's output and other information.
- Dense MoE: Use all experts' outputs and mix based on the gate's probabilities.
- Sparse MoE: Inputs are only passed to the experts with the highest gate probabilities (e.g. top-2). Outputs are a weighted sum (rescaled) of the chosen experts' outputs.

$$\begin{bmatrix} -\infty & -\infty & -\infty & -\infty & -\infty & -\infty \\ -\infty & -\infty & -\infty & -\infty & -\infty & -\infty \\ -\infty & -\infty & -\infty & -\infty & -\infty & -\infty \\ -\infty & -\infty & -\infty & -\infty & -\infty & -\infty \end{bmatrix}$$

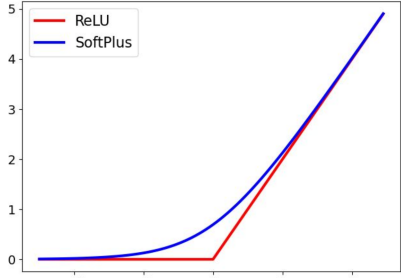


$$\begin{bmatrix} -\infty & -\infty & 0.205 & -\infty & 0.211 & -\infty \\ 0.203 & -\infty & -\infty & -\infty & 0.206 & -\infty \\ -\infty & -\infty & 0.213 & -\infty & 0.206 & -\infty \\ 0.220 & -\infty & -\infty & -\infty & 0.212 & -\infty \end{bmatrix}$$



$$\begin{bmatrix} 0 & 0 & 0.498 & 0 & 0.502 & 0 \\ 0.499 & 0 & 0 & 0 & 0.501 & 0 \\ 0 & 0 & 0.502 & 0 & 0.498 & 0 \\ 0.502 & 0 & 0 & 0 & 0.498 & 0 \end{bmatrix}$$

- Limitations:
- Gating mechanism often selects only the most prominent experts. Model will be underutilized, may degrade overall performance of model. Reduces diversity in learning and limits the specialization of different experts.
- Stochastic routing:
 - Gives all experts a chance to be selected early in training: (passive) load balancing
 - Adding of noise can be a tunable operation by introducing additional learnable parameters
 - $G(x) = \text{Softmax}(\text{TopK}(H(x), k)) H(x) = (\mathbf{W}_{gate}\mathbf{x}) + \text{StandardNormal}() \cdot \text{SoftPlus}(\mathbf{W}_{noise}\mathbf{x})$
- SoftPlus:



- Smooth and differentiable approximation of ReLU, makes optimization more stable and always positive.
- Load balancing: Common strategy: auxiliary loss to penalize an imbalance utilization of experts (but other strategies have been proposed as well)

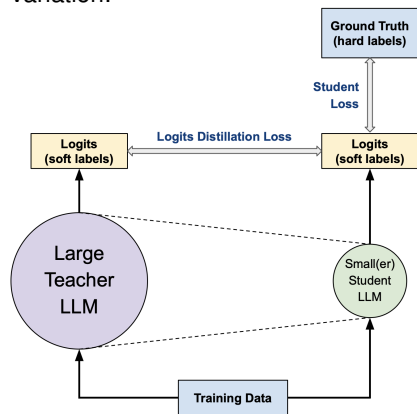
$$L_{aux} = w_{aux} \cdot N \cdot \sum_{i=1}^n f_i \cdot P_i$$

$$f_i = \frac{1}{T} \sum_{\mathbf{x} \in \mathcal{B}} \mathbb{1}\{\text{argmax } p(\mathbf{x}) = i\}$$
$$P_i = \frac{1}{T} \sum_{\mathbf{x} \in \mathcal{B}} p_i(\mathbf{x})$$

Fraction of samples routed to Expert i fraction of router probabilities allocated to Expert i

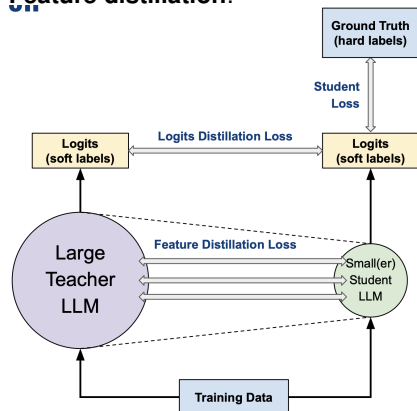
Distillation

- Motivation: Large LLMs are expensive to train and run.
- **Response distillation:** Teacher LLM labels the training data and the student LLM learns from the teacher's labels.
Limitations: good teacher & dataset needed, expensive, no nuanced knowledge, less creative.
- **Logit distillation:**
 1. Give same data to teacher and student.
 2. Minimize the KL divergence (**distillation loss**) between the teacher and student logits.
- Advantages: no need for labelled data, nuances captured through logits.
- Disadvantages: teacher's biases inherited by student, missing ground truth for evaluation.
- Variation:



- Also calculate the "normal" loss based on the labelled dataset.
- Advantages: richer training signal, more stable training, straightforward evaluation.
- Limitations: added complexity, conflicting signals, non-obvious combination of losses.

Feature distillation:



- Student learns to mimic the teacher's intermediate

representations.

- Limitations: non-trivial layer alignment for different architectures.
- **Question:** Why logits but not softmax probabilities? It is because the raw logits better preserve the relative confidence of the teacher model.